IEEE 754 Floating-Point Representation and Arithmetic

Lecture Notes — Week 3

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

August 8, 2026

3.1 Introduction

Week 2 built the integer layer: two's complement representation, fixed point, and the adder hardware that operates on them. But integers cover a small, fixed range. Real numbers in science span from roughly 10^{-300} to 10^{300} — a single fixed point cannot cover them. This chapter is about **IEEE 754** floating point, the representation that trades a little precision for a huge range by storing a number as sign, exponent, and mantissa. It answers four questions that form one thread: the scientific-notation idea floating point is built on (Section 3.2); the IEEE 754 single-precision format, bit by bit (Section 3.3); normalization, special values, and rounding (Section 3.4); and how add/multiply actually run, where precision is lost (Section 3.5).

Throughout the chapter we thread a single worked example: encoding **10.75** in IEEE 754 single precision, then adding it to another number and watching where precision gets lost. The thread is closed in Section 3.6 when the encoding is checked and the arithmetic traced.

3.2 Scientific Notation

3.2.1 Motivation: One Sign, One Mantissa, One Exponent

The value 10^{-30} has a single significant "1" buried under thirty leading zeros. Scientific notation writes it as 1.0×10^{-30} — three fields that carry the whole meaning: **sign**, **mantissa/significand** (the digits), and **exponent** (where the point goes). A fixed-point representation would need hundreds of bits to place that "1"; floating point needs only the mantissa's precision. The range comes from the exponent, the precision from the mantissa — independent knobs.

Definition (Floating-Point Value). *A floating-point number represents* $(-1)^S \times M \times r^E$*, where* S *is the* ***sign*** *bit,* M *the* ***mantissa****,* E *the* ***exponent****, and* r *the base (2 in hardware). In base 2, a normalized value is* $1.xxx_2 \times 2^E$ *(P&H, Sec. 3.5, p.205; Hamacher Ch. 9) [2, 1].*

Example 3.2.1 (Binary scientific notation)**.** $10.75 = 8 + 2 + 0.5 + 0.25 = 1010.11_2$*. Normalizing gives* $1010.11_2 = 1.01011_2 \times 2^3$*: sign* $S = 0$*, mantissa* $= 01011$*, exponent* $E = 3$*.*

3.2.2 Socratic Check: What Does Normalization Buy?

Any binary number has many representations ($10.75 = 0.101011 \times 2^4 = 1.01011 \times 2^3$). If we agree the mantissa always starts with a leading 1 ("normalized"), we gain a **unique** representation for every value, and we can **omit the leading 1** — it is implicit — gaining one extra bit of precision for free. This hidden "1." in front of the stored mantissa is exactly what IEEE 754 does.

3.3 IEEE 754 Single Precision

3.3.1 The 32-Bit Layout

IEEE 754 single precision packs the three fields into 32 bits (Figure 3.1): 1 sign bit, 8 exponent bits, 23 mantissa bits. The value is

$$(-1)^S \times (1.M)_2 \times 2^{E-\text{bias}},$$

where the mantissa field M stores only the fraction (the leading 1 is hidden), and the **bias** is 127 for single precision: the stored exponent is $E_{\text{stored}} = E_{\text{true}} + 127$ (P&H, Sec. 3.5, p.205) [2].

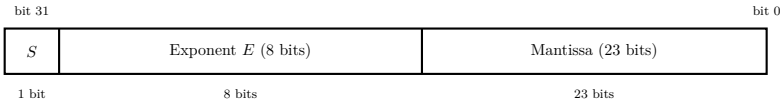

Figure 3.1: IEEE 754 single precision: 1 sign + 8 exponent + 23 mantissa, value $= (-1)^S \times (1.M)_2 \times 2^{E-127}$.

3.3.2 Why Bias the Exponent?

The true exponent can be negative ($0.5 = 1.0 \times 2^{-1}$ has $E = -1$). Storing it as two's complement would complicate ordering. The **bias** solves this: store $E_{\text{stored}} = E_{\text{true}} + 127$ as an unsigned number. The result is that the all-zeros exponent means "smallest" and all-ones "largest," so unsigned comparison correctly orders floats. For single precision, E_{true} ranges from -126 to $+127$; the extreme all-0 and all-1 exponent patterns are reserved for special values (Section 3.4).

Example 3.3.1 (Storing the exponent of 10.75)**.** *10.75 has* $E_{true} = 3$*, so* $E_{stored} = 3 + 127 = 130 = 1000\,0010_2$.

Example 3.3.2 (Encoding 10.75)**.** *From Section 3.2:* $10.75 = 1.01011_2 \times 2^3$*. Sign* $S = 0$*; mantissa field* $M = 01011\,0000\,0000\,0000\,0000\,000$ *(23 bits); exponent* $E_{stored} = 1000\,0010_2$*. The full 32 bits are*

$$\underbrace{0}_{sign}\ \underbrace{1000\,0010}_{exp=130}\ \underbrace{01011000000000000000000}_{mantissa},$$

i.e. `0x412C0000`*. Check:* $1.01011_2 \times 2^{130-127} = 1.01011_2 \times 2^3 = 1010.11_2 = 10.75$.

3.3.3 Socratic Check: Decoding Back

Decode `0x3F800000`: sign 0, exponent $0111\,1111_2 = 127$, mantissa all zeros. Then $E_{\text{true}} = 127 - 127 = 0$ and value $= 1.0 \times 2^0 =$ **1.0**. The bias is chosen so that **1.0** is the most ordinary pattern (exponent 0111 1111) — the "anchor" number, useful for sanity-checking any encoding by how far the exponent field is from 127.

3.4 Normalization, Special Values, and Rounding

3.4.1 Special Values: When the Format Runs Out

The reserved exponent patterns give the format well-defined behavior at its extremes (P&H, Sec. 3.5, p.205) [2]. **Zero**: exponent all 0, mantissa all 0 (+0 and −0). **Denormal (subnormal)**: exponent

all 0, mantissa nonzero — numbers smaller than the smallest normalized value, for which the leading 1 is *not* assumed. **Infinity**: exponent all 1, mantissa all 0 ($\pm\infty$). **NaN** (Not a Number): exponent all 1, mantissa nonzero — produced by $0/0$, $\sqrt{-1}$, and similar undefined operations.

Example 3.4.1 (The smallest and largest normalized single-precision values)**.** *Smallest normalized: exponent* 1*, mantissa 0* $\Rightarrow 1.0 \times 2^{-126} \approx 1.18 \times 10^{-38}$*. Largest normalized: exponent* 254*, mantissa all 1* $\Rightarrow (2 - 2^{-23}) \times 2^{127} \approx 3.4 \times 10^{38}$*. The range is roughly* 10^{-38} *to* 10^{38} *— far beyond fixed point — but precision is fixed at about 7 significant decimal digits (the 24-bit mantissa including the hidden 1).*

3.4.2 Rounding and Precision Loss

Most real numbers are *not* exactly representable — 0.1 repeats in binary. IEEE 754 rounds to the nearest representable value (ties to even). Precision is 24 bits of mantissa $\approx$ 7 decimal digits; rounding error is at most $\frac{1}{2}$ ULP (unit in the last place). The practical consequence: **floating-point arithmetic is approximate**, and equality tests on floats are dangerous.

Example 3.4.2 (Where precision is lost)**.** *Add* $10.75 + 0.1$ *in single precision.* 0.1 *is stored as the approximation* $0.1 \approx 1.10011001100110011001101_2 \times 2^{-4}$*, rounded to 24 bits. Adding a* large *and a* small *number pushes the small one's low bits out of the 24-bit window:* $10.75 + 0.1$ *rounds to about* 10.85*, but* $1\,000\,000 + 0.1$ *rounds to* $1\,000\,000$ *— the* 0.1 *vanishes entirely. Precision is lost at the* least significant *digits, and the loss is worst when adding numbers of very different magnitudes.*

3.4.3 Socratic Check: Why Not Compare Floats for Equality?

In a loop you compute $x = 0.1$ ten times by addition. Is $x == 1.0$ guaranteed true? No — each 0.1 is an approximation, and ten additions accumulate rounding error, so x comes out slightly off 1.0. The rule: compare floats with a **tolerance** ($|a - b| < \epsilon$), never with ==. This is the practical consequence of finite precision.

3.5 Floating-Point Arithmetic

3.5.1 Motivation: Adding Two Exponents

To add $1.5 \times 2^3 + 1.25 \times 2^5$, you cannot just add the mantissas — their exponents differ. The hardware must first **align exponents**: shift the smaller mantissa right until both exponents match, then add the mantissas, re-normalize, and round. The alignment step is exactly where the low-bit loss of the previous section comes from.

3.5.2 The Floating-Point Add Algorithm

IEEE 754 addition is four steps (P&H, Sec. 3.5, p.205) [2]:

1. **Align**: shift the mantissa of the smaller-exponent operand right until exponents are equal (track the shifted-out bits).
2. **Add**: add the (aligned) mantissas.
3. **Normalize**: shift the result to have a leading 1; adjust the exponent.
4. **Round**: round the mantissa to the stored width (ties to even).

Multiplication is easier: multiply mantissas, add exponents (subtract the bias once), normalize, round — no alignment step.

Example 3.5.1 (Adding 10.75 + 0.1 (qualitative))**.** $10.75 = 1.01011_2 \times 2^3$*;* $0.1 \approx 1.100110\ldots_2 \times 2^{-4}$*. Align: shift* 0.1*'s mantissa right by* $3-(-4)=7$ *places. Add mantissas, re-normalize, round to 24 bits. The result is approximately* 10.85*, but the alignment* ***truncated*** *low bits of 0.1, so the sum is slightly off the exact value. The error is invisible in the 7-digit window — precision loss is* ***silent****.*

3.5.3 Socratic Check: Multiplication, One Line

The product of two normalized floats: sign $S = S_1 \oplus S_2$; mantissa $M = M_1 \times M_2$ (re-normalized); exponent $E = E_1 + E_2 - \text{bias}$. No alignment is needed (exponents just add), but the mantissa product is wider, requiring normalization and rounding.

3.6 Synthesis and Summary

3.6.1 The Thread, Closed

The chapter's running thread encoded **10.75** as `0x412C0000` and then added 0.1, watching the alignment step truncate low bits of the smaller operand — silent precision loss. The encoding used every field of the format (sign, exponent with bias, mantissa with the hidden 1), and the arithmetic used every step of the add algorithm (align, add, normalize, round).

3.6.2 Bridge to Week 4

We now know how numbers are represented (integers, fixed point, IEEE 754) and how the ALU computes on them. But bits and adders do not run a program by themselves. Week 4 builds **the machine**: registers (the fast scratchpad), the stack (LIFO operand home), instruction formats, and the fetch–decode–execute cycle that animates them — the storage spaces and the cycle.

3.6.3 Summary

- **Floating point** $= (-1)^S \times 1.M \times 2^{E-\text{bias}}$: range from the exponent, precision from the mantissa.
- **Single precision:** 32 bits $= 1 + 8 + 23$, bias 127, hidden 1.
- **Special values:** zero/denormal/infinity/NaN from reserved exponent patterns.
- **Add:** align, add, normalize, round. **Multiply:** multiply mantissas, add exponents, normalize, round.
- **Precision:** about 7 digits; rounding is silent and approximate — never compare floats with ==.

References

[1] V. Carl Hamacher, Zvonko G. Vranesic, and Safwat G. Zaky. *Computer Organization*. McGraw-Hill, 5th edition, 2002.

[2] David A. Patterson and John L. Hennessy. *Computer Organization and Design: The Hardware/Software Interface*. Morgan Kaufmann, arm edition (5th) edition, 2017.